

SI1001 Teoría de la Computación

Verificación de programas: Introducción

Andrés Sicard Ramírez

Universidad EAFIT

Semestre 2025-2

Cinco preguntas

En el prefacio de (Hein 2017) el autor señala que la mayoría de los problemas en Ciencias de la Computación involucran una de las siguientes cinco preguntas:

- (i) *«Can the problem be solved by a computer program?»*
- (ii) *«If not, can you modify the problem so that it can be solved by a program?»*
- (iii) *«If so, can you write a program to solve the problem?»*
- (iv) *«Can you convince another person that your program is correct?»*
- (v) *«Can you convince another person that your program is efficient?»*

Antecedentes

- «*Checking a Large Routine*» (Turing, 1949)
- «*A Basis for a Mathematical Theory of Computation*» (McCarthy, 1963)
- «*Proof of Algorithms by General Snapshots*» (Naur, 1966)
- «*A Constructive Approach to the Problem of Program Correctness*» (Dijkstra, 1967)
- «*Assigning Meanings to Programs*» (Floyd, 1967)
- «*An Axiomatic Basis for Computer Programming*» (Hoare, 1969)
- LCF (*Logic for Computable Functions*) (Scott, 1969, 1993)
- «*Procedures and Parameters: An Axiomatic Approach*» (Hoare, 1971)

Clasificación (paradigmas) de los lenguajes de programación

Clasificación

- Declarativos (¿qué?)
 - Funcionales (LISP, ML, HASKELL, ...)
 - Lógicos, basados en restricciones (PROLOG, CLP, ...)
- Imperativos (¿cómo?)
 - von Neumann (FORTRAN, PASCAL, C, ADA, ...)
 - Orientados a objetos (SMALLTALK, C++, JAVA, ...)
 - *Scripting* (PERL, PYTHON, PHP, ...)

Clasificación (paradigmas) de los lenguajes de programación

Clasificación^a

- Declarativos (¿qué?)
 - Funcionales (LISP, ML, HASKELL, ...)
 - Lógicos, basados en restricciones (PROLOG, CLP, ...)
- Imperativos (¿cómo?)
 - von Neumann (FORTRAN, PASCAL, C, ADA, ...)
 - Orientados a objetos (SMALLTALK, C++, JAVA, ...)
 - *Scripting* (PERL, PYTHON, PHP, ...)

^aClasificación adaptada de la clasificación presentada en (Scott 2015).

La arquitectura de von Neumann

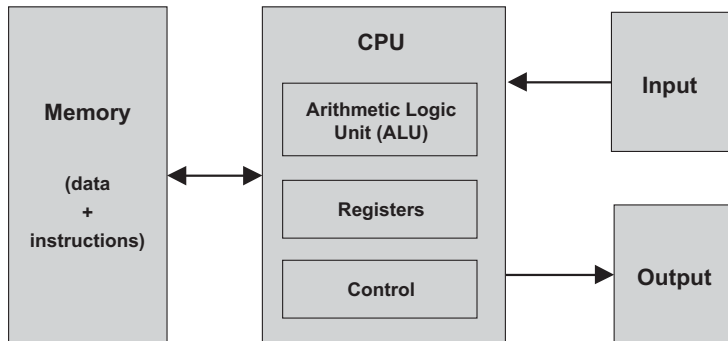


Figura 5.1 en (Nisan y Shimon 2005).

Programas en lenguajes de máquina

Ejemplo

Programa en lenguaje de máquina que adiciona los números 5 y 6.

```
0010001000000100
0010010000000100
0001011001000010
0011011000000011
1111000000100101
0000000000000101
0000000000000110
```

Figura 1.2 en (Louden y Lambert 2011).

Programas en lenguajes ensambladores

Ejemplo

Programa en lenguaje ensamblador que adiciona los números en las variables `FIRST` y `SECOND` y almacena el resultado en la variable `SUM`.

```
.ORIG x3000      ; Address (in hexadecimal) of the first instruction
LD  R1, FIRST    ; Copy the number in memory location FIRST to register R1
LD  R2, SECOND   ; Copy the number in memory location SECOND to register R2
ADD R3, R2, R1    ; Add the numbers in R1 and R2 and place the sum in
                  ; register R3
ST  R3, SUM       ; Copy the number in R3 to memory location SUM
HALT              ; Halt the program
FIRST  .FILL #5   ; Location FIRST contains decimal 5
SECOND .FILL #6   ; Location SECOND contains decimal 6
SUM     .BLKW #1  ; Location SUM (contains 0 by default)
.END            ; End of program
```

Figura 1.3 en (Louden y Lambert 2011).

Programas en lenguajes imperativos

Ejemplo

Programa en lenguaje C que adiciona los números en las variables `first` y `second` y almacena el resultado en la variable `sum`.

```
int first = 5;  
int second = 6;  
sum = first + second;
```

Programas en lenguajes imperativos

Descripción

Un programa en un lenguaje imperativo es una secuencia de instrucciones que representan comandos:

- instrucciones de asignación,
- instrucciones de secuenciación,
- instrucciones *if-then-else*,
- instrucciones *while*.

Razonamiento sobre programas

Categorías

La investigación acerca el razonamiento sobre programas se puede dividir en cuatro categorías (Achten et al. 2010):

- (i) Definir la semántica (nuevos conceptos).
- (ii) Razonamiento sobre transformaciones de programas.
- (iii) **Verificación (formal) de propiedades funcionales** (correspondencia de entrada y salida).
- (iv) Verificación (formal) de propiedades no funcionales (consumo de memoria, rendimiento paralelo, etc.).

Correctitud de programas

Correctitud parcial y correctitud total

Al establecer la correctitud funcional (correspondencia de entrada y salida) de un programa se distingue entre:

- **Correctitud parcial:** El programa es correcto respecto a su especificación.
- **Correctitud total:** Correctitud parcial + terminación.

Correctitud de programas

Observación

*«In contrast to popular belief, proving termination **is not always impossible**».*
(Cook, Podelski y Rybalchenko **2011**, pág. 1)

Algunos sistemas (herramientas) para la verificación de programas

Para lenguajes específicos

- CakeML (implementación verificada de ML).
- CompCert (implementación verificada de un subconjunto de C).
- Frama-C y su plugin WP (programas en C).
- KeY (programas en JAVA)

Algunos sistemas (herramientas) para la verificación de programas

Para lenguajes específicos

- CakeML (implementación verificada de ML).
- CompCert (implementación verificada de un subconjunto de C).
- Frama-C y su plugin WP (programas en C).
- KeY (programas en JAVA)

Sistemas generales

- Agda.
- Idris.
- Isabelle/HOL.
- HOL4, HOL Light.
- Rcoq (antes Coq).
- Why3.

Referencias

-  Peter Achten, Marko van Eekelen, Pieter Koopam y Marco T. Morazán (2010). Trends in *Trends in Functional Programming* 1999/2000 versus 2007/2008. Higher-Order Symbolic Computation 23.4, págs. 465-487. DOI: [10.1007/s10990-011-9074-z](https://doi.org/10.1007/s10990-011-9074-z) (vid. pág. 11).
-  Byron Cook, Andreas Podelski y Andrey Rybalchenko (2011). Proving Program Termination. Communications of the ACM 54.5, págs. 88-98. DOI: [10.1145/1941487.1941509](https://doi.org/10.1145/1941487.1941509) (vid. pág. 13).
-  Robert W. Floyd (1967). Assigning Meanings to Programs. En: Mathematical Aspects of Computer Science. Ed. por Jacob T. Schwartz. Vol. 19. Proceedings of Symposia in Applied Mathematics. 1967, págs. 19-32 (vid. pág. 3).
-  James L. Hein (2017). Discrete Structures, Logic, and Computability. 4.^a ed. Jones & Bartlett Learning, 2017 (1995) (vid. pág. 2).
-  C. A. R. Hoare (1969). An Axiomatic Basis for Computer Programming. Communications of the ACM 12.10, 576-580(3). DOI: [10.1145/363235.363259](https://doi.org/10.1145/363235.363259) (vid. pág. 3).

Referencias

-  C. A. R. Hoare (1971). Procedures and Parameters: An Axiomatic Approach. En: Symposium on Semantics of Algorithmic Languages. Ed. por E. Engler. Vol. 188. Lecture Notes in Mathematics. Springer Berlin Heidelberg, 1971, págs. 102-116. DOI: [10.1007/BFb0059696](https://doi.org/10.1007/BFb0059696) (vid. pág. 3).
-  Kenneth C. Loudon y Kenneth A. Lambert (2011). Programming Languages. Principles and Practice. 3.^a ed. Cengage Learning, 2011 (1993) (vid. págs. 7, 8).
-  John McCarthy (1963). A Basis for a Mathematical Theory of Computation. En: Computer Programming and Formal Systems. Ed. por P. Braffort y D. Hirshberg. North-Holland, 1963, págs. 33-70 (vid. pág. 3).
-  Peter Naur (1966). Proof of Algorithms by General Snapshots. BIT 6.4, págs. 310-316 (vid. pág. 3).
-  Noam Nisan y Schocken Shimon (2005). The Elements of Computing Systems. Building a Modern Computer from First Principles. MIT Press, 2005 (vid. pág. 6).
-  Michael L. Scott (2015). Programming Language Pragmatics. 4.^a ed. Morgan Kaufmann, 2015 (1999) (vid. pág. 5).

Referencias



Alan Turing (1949). Checking a Large Routine. En: Report of a Conference on High Speed Automatic Calculating Machines. 1949, págs. 67-69 (vid. [pág. 3](#)).